



SE507 - SOFTWARE TESTING AND EVALUATION

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah (Test Manager)
Sara Zuhair (Code specifier)
Yassin Hussein (Code reviewer)
Ravyar Barzan (Document inspector)

Module Leader: **Dr. Hossein Hassani**

June 14, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

List of Figures

List of Tables

Background

This chapter provides a comprehensive overview of the Library Management System, the System Under Test (SUT). It details the system's purpose, architecture, user roles, core functionalities, and underlying data model. This context is essential for understanding the scope, strategy, and execution of the testing activities detailed in subsequent chapters.

1.1 System Under Test (SUT) Overview

The Library Management System is a full-stack web application designed to modernize and streamline library operations **srs**. Its primary purpose is to provide a digital platform for managing library resources, including books, authors, and genres, while also managing user accounts for both librarians and library members **srs**. The system aims to replace or supplement traditional, manual library management methods with an efficient, accessible, and user-friendly web-based solution **srs**.

The project's scope encompasses a complete, self-contained system featuring a backend API, a database, and a frontend client application **srs**. Key capabilities include:

- **User Authentication:** Secure user registration, login, and logout functionalities with role-based access control for two distinct user types: Librarians and Members **srs**.
- **Inventory Management:** Comprehensive CRUD (Create, Read, Update, Delete) operations for the core library entities: books, authors, and genres **srs**.
- **Borrowal Tracking:** Management of book borrowals and returns, allowing librarians to oversee lending activities and members to view their history **srs**.
- **User Administration:** Tools for librarians to manage all user accounts within the system **srs**.
- **Review System:** A newly introduced feature allowing members to submit reviews for books, which can then be managed by librarians **srs**.

1.2 System Architecture

The system is built on a modern, multi-tier architecture, which separates concerns between the client, server, and database. This design promotes modularity, scalability, and maintainability.

- **Frontend (Client-Side):** The user interface is a single-page application (SPA) developed using **React** and the **Material-UI (MUI)** component library. It is responsible for rendering the UI and interacting with the backend via API calls. It runs in any modern web browser **srs**.

- **Backend (Server-Side):** The backend is a RESTful API built with **Node.js** and the **Express.js** framework. It handles all business logic, data processing, and user authentication using Passport.js **srs**.
- **Database:** Data persistence is managed by a **MongoDB** database, a NoSQL database that stores data in flexible, JSON-like documents. The backend interacts with MongoDB via the Mongoose ODM library **srs**.

The entire application is containerized using **Docker**, ensuring consistent development, testing, and deployment environments **srs**. Figure ?? illustrates the high-level architecture.

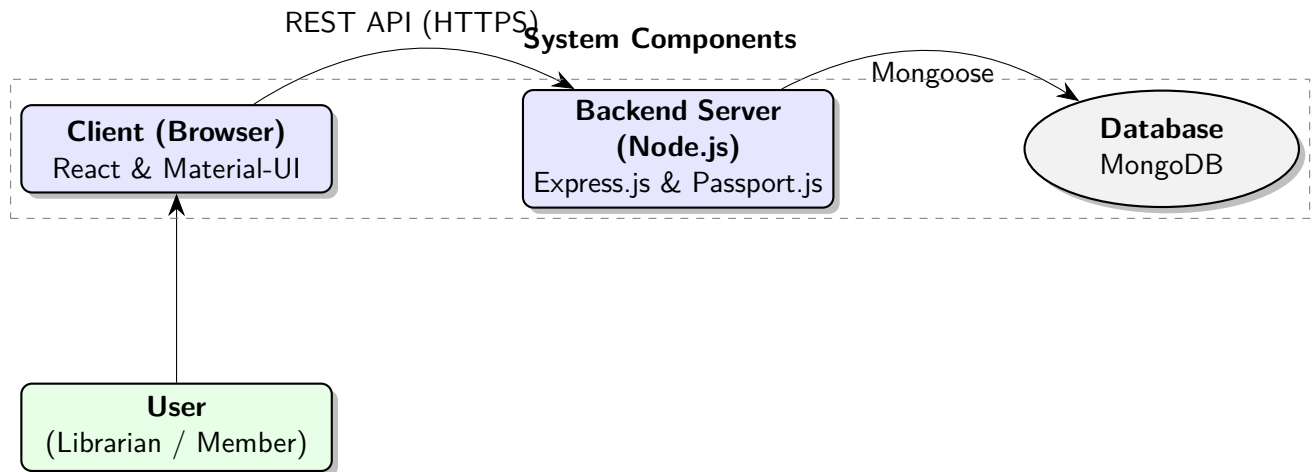


Figure 1.1: High-Level System Architecture.

1.3 User Roles and Functionalities

The system defines two distinct user roles, each with a specific set of permissions and responsibilities as outlined in the SRS **srs**.

1.3.1 Librarian (Admin)

Librarians are the administrators of the system with full control over all its aspects. Their primary responsibilities include:

- Managing the library's entire inventory, including adding, updating, and deleting books, authors, and genres **srs**.
- Overseeing all user accounts, which involves creating new accounts for both members and other librarians, as well as updating and deleting existing accounts **srs**.
- Managing the complete lifecycle of book borrowals, from creation to marking returns **srs**.
- Viewing system-wide statistics and data on a dedicated dashboard **srs**.
- Moderating book reviews submitted by members **srs**.

1.3.2 Member

Members are the standard users of the library. Their access is generally limited to viewing data and managing their own activities. Their functions include:

- Browse the library's catalog of books, authors, and genres **srs**.

- Requesting to borrow available books **srs**.
- Viewing their personal borrowal history **srs**.
- Submitting, viewing, and managing their own reviews for books **srs**.

1.4 Data Model

The system's data is structured around several core entities, whose relationships define the application's functionality. The main entities are **User**, **Book**, **Author**, **Genre**, **Borrowal**, and **Review srs**. The relationships between these entities are a key focus of integration testing to ensure data integrity. For instance, a *Borrowal* record links a *User* to a *Book*, and a *Book* can have one *Author* and one *Genre srs*.

Figure ?? provides a conceptual Entity-Relationship Diagram (ERD) illustrating these connections.

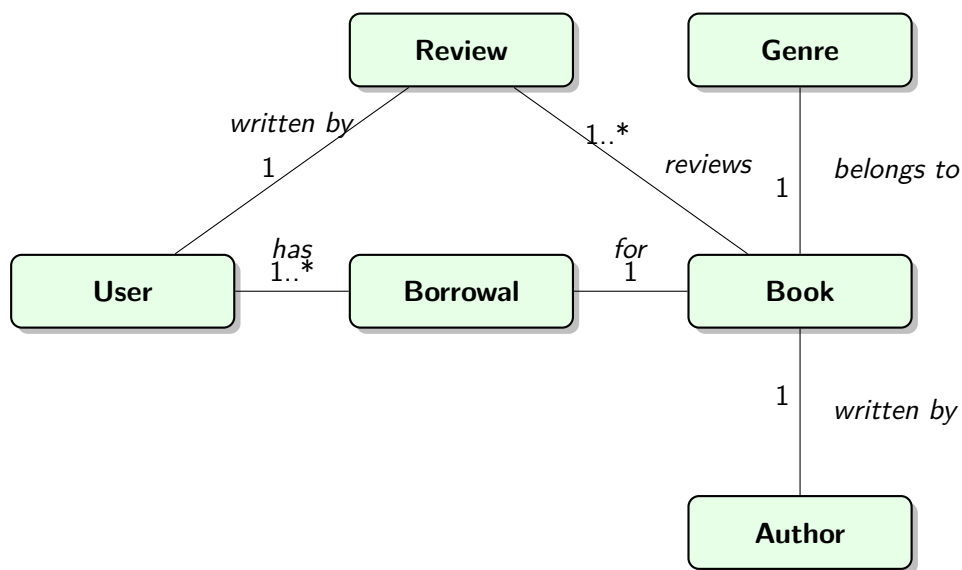


Figure 1.2: Conceptual Entity-Relationship Diagram (ERD).

Test Management

2.1 Introduction

This chapter outlines the comprehensive management plan for the testing of the Library Management System. The plan is designed to ensure that all software components are rigorously tested, meet the requirements specified in the Software Requirements Specification (SRS), and achieve the highest standards of quality. This document details the test strategy, scope, team roles, environment, schedule, deliverables, risk management, and success criteria for the project.

2.2 Test Strategy

Our test strategy employs a multi-layered approach, integrating various testing levels to ensure comprehensive coverage and early defect detection. The strategy is aligned with the V-Model, ensuring that testing activities are planned in parallel with development phases.

2.2.1 Overall Approach

The testing process is structured across multiple levels:

- **Static Testing:** Performed without executing the code, focusing on reviews and analysis of documentation and source code to find defects early.
- **Unit Testing:** Developers will test individual components (functions, classes) in isolation to verify their correctness.
- **Integration Testing:** Focuses on testing the interfaces and interactions between integrated components, such as the communication between the frontend client and the backend API.
- **System & E2E Testing:** The complete, integrated system is tested from end-to-end to validate that it meets all specified requirements from a user's perspective.
- **Non-Functional Testing:** Ensures the system meets requirements for performance, security, usability, and compatibility.
- **Regression Testing:** Conducted after modifications to ensure that new changes have not introduced new defects into existing functionality.

2.2.2 Test Levels

- **Unit Testing:** Implemented using **Jest**. For the backend, this involves testing Node.js controllers, models, and utility functions. For the frontend, this includes testing React components, hooks, and utility functions using Jest and the React Testing Library.

- **API & Integration Testing:** Backend API endpoints are tested using **Jest** and **Supertest** to verify CRUD operations, authentication flows, and business logic. This also covers the integration points between different services.
- **End-to-End (E2E) Testing:** Real user workflows are simulated using **Cypress**. These tests cover complete user journeys, such as logging in, searching for a book, borrowing it, and logging out.
- **Non-Functional Testing:**
 - **Performance Testing:** **K6** is used to script and execute load tests against the API to measure response times, throughput, and stability under load.
 - **Security Testing:** Manual and automated checks are performed to identify common vulnerabilities (e.g., SQL injection, XSS, insecure direct object references). Cypress tests are used to check for authorization flaws.
 - **Usability & Compatibility Testing:** Manual testing and Cypress scripts ensure the application is user-friendly and renders correctly across major web browsers (Chrome, Firefox, Safari).

2.3 Test Scope

2.3.1 In Scope

The following features and functionalities, as defined in the SRS, are within the scope of testing:

- User Authentication (Login, Logout, Role-based Access Control)
- Librarian Dashboard and statistical widgets
- Full CRUD (Create, Read, Update, Delete) operations for Books, Authors, and Genres
- Borrowal Management (borrowing, returning, and tracking books)
- User Management (Librarian managing member accounts)
- Book Review and Rating system
- API endpoint functionality and security
- Database referential integrity

2.3.2 Out of Scope

The following are considered out of scope for testing:

- Testing of third-party libraries and frameworks (e.g., React, Express, Mongoose).
- Performance of the underlying hardware, network infrastructure, or MongoDB database engine itself.
- Exhaustive testing of every possible combination of user inputs.
- Usability testing on mobile operating systems (unless specified).

2.4 Team Roles and Responsibilities

The testing process involves the entire team, with specific roles and responsibilities as defined in the project's task distribution plan.

Table 2.1: Team Roles and Primary Responsibilities

Member	Role	Primary Testing Responsibilities
Tishko Salah	Test Manager	Oversees the entire testing process, manages the test plan and schedule, coordinates team efforts, and is the final authority on test completion.
Sara Zuhair	Code Specifier	Focuses on non-coder testing activities, including functional validation against SRS, documentation review, and usability testing. Leads the execution of user-centric test cases.
Yassin Hussein	Code Reviewer	Leads technical testing activities, including unit testing, API testing, and code reviews. Ensures code quality, performance, and adherence to standards.
Ravyar Barzan	Document Inspector	Leads the review and inspection of all project documentation (SRS, Test Plan, Test Cases, Final Report) for clarity, consistency, and completeness.

2.5 Test Environment and Tools

All testing is conducted within containerized environments managed by Docker to ensure consistency and reproducibility.

- **Operating Systems:** Windows, macOS, and Linux (for development and execution).
- **Test Environments:** Defined in `docker-compose.dev.yml`, `docker-compose.test.yml`, and `docker-compose.prod.yml` to simulate development, testing, and production setups.
- **Version Control:** Git, with GitHub for repository hosting and collaboration.

Table 2.2: Testing Tools and Frameworks

Category	Tool/Framework	Purpose
Backend Testing	Node.js, Jest, Supertest	Unit and integration testing of API endpoints and services.
Frontend Testing	Jest, React Testing Library	Unit and component testing of React components and logic.
E2E Testing	Cypress	Automated end-to-end testing of user workflows in a browser.
Performance	K6, Docker	Load testing the backend API to measure performance metrics.
Code Quality	ESLint, Prettier	Static code analysis to enforce coding standards and find bugs.
Documentation	LaTeX, Draw.io	Creating project reports, diagrams, and test artifacts.

2.6 Test Schedule

The project timeline spans approximately 14 weeks, from early March to mid-June 2025. The schedule is visualized in the Gantt chart below (Figure ??).

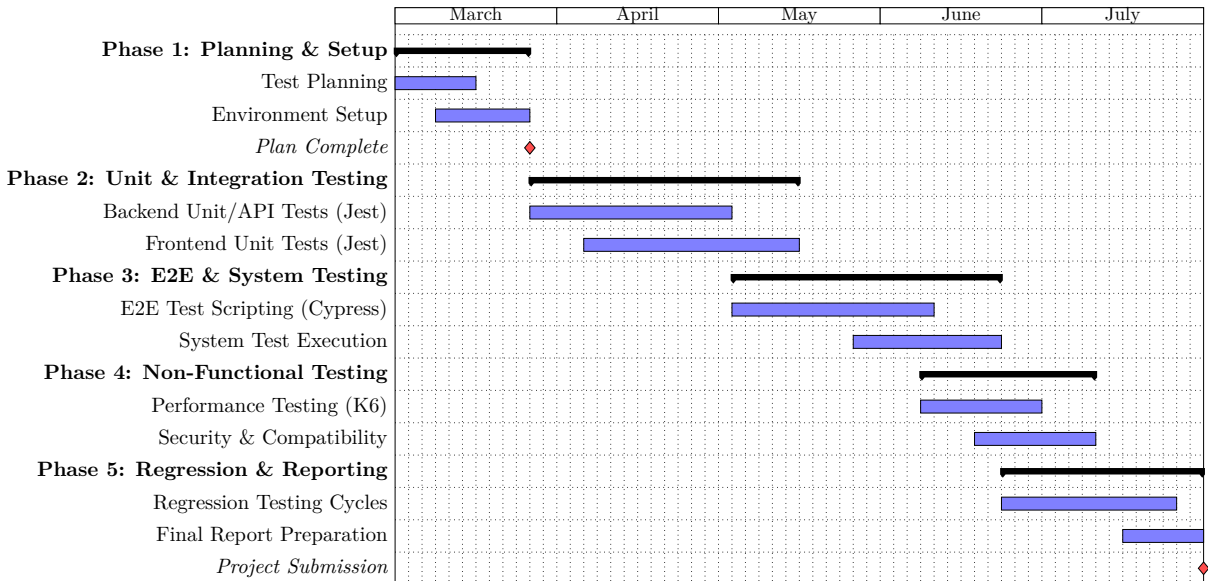


Figure 2.1: Project Testing Schedule Gantt Chart

2.7 Deliverables

The following artifacts will be produced as part of the testing process:

- **Test Plan:** This comprehensive document (of which this chapter is a part).
- **Test Case Documentation:** Detailed test cases for all levels of testing, located in `/docs/Report/test_artifacts/test_cases/`.
- **Test Scripts:** All automated test scripts written using Jest, Cypress, and K6.
- **Defect Reports:** Bugs and issues will be tracked, prioritized, and managed. (A system like GitHub Issues will be used).
- **Test Execution Reports:** Automated reports generated by Jest (`/server/test-report/report.html`) and Cypress (videos/screenshots) after test runs.
- **Final Test Summary Report:** The complete final report detailing all testing activities, results, and conclusions.

2.8 Risk Management

Potential risks that could impact the testing schedule and quality are identified and managed proactively.

Table 2.3: Risk Assessment and Mitigation Plan

Risk Description	Likelihood	Impact	Mitigation Strategy
Delays in development phases impact testing start dates.	Medium	High	Maintain close communication between developers and testers. Adapt the test plan and prioritize testing of critical features.
Test environment is unstable or differs from production.	Low	High	Use Docker to ensure consistent environments. Perform smoke tests on deployment to validate environment stability.
Critical defects are discovered late in the project timeline.	Medium	High	Emphasize early testing (unit, integration). Prioritize test cases based on risk and critical user workflows.
Team member unavailability due to unforeseen circumstances.	Low	Medium	Ensure knowledge is shared across the team. Cross-train members on critical testing tasks and tools.
Underestimation of time required for test case creation or execution.	Medium	Medium	Base estimates on feature complexity. Regularly review progress and re-allocate resources as needed.

2.9 Entry and Exit Criteria

Clear criteria define the start and end points for the formal testing process.

2.9.1 Entry Criteria

Testing on a feature or build can begin when:

- The SRS and relevant design documents are complete and approved.
- The code has been developed, peer-reviewed, and successfully built.
- All developer-written unit tests for the feature are passing.
- The required test environment and test data are available and stable.
- Test cases have been written, reviewed, and approved by the test manager.

2.9.2 Exit Criteria

The overall testing phase will be considered complete when:

- All planned test cases have been executed.
- A pass rate of 100% for all critical and major test cases is achieved.
- A pass rate of over 95% for minor test cases is achieved.
- No outstanding Severity-1 (blocker) or Severity-2 (critical) defects remain open.

- All major functional and non-functional requirements are verified and met.
- The final test report is completed and approved by all team members.

Testing Process

3.1 Introduction

The testing process for the Library Management System is a structured and multi-layered approach designed to ensure the delivery of a high-quality, reliable, and robust application. Its primary goal is to systematically verify and validate that the system meets the functional and non-functional requirements detailed in the Software Requirements Specification (SRS). This process is integral to mitigating risks, identifying defects early in the development lifecycle, and ultimately ensuring user satisfaction.

Our methodology emphasizes automation to facilitate a continuous integration and testing pipeline. By leveraging containerization with Docker, we establish consistent and isolated environments for development, testing, and production. This is crucial for achieving reliable and reproducible test outcomes, eliminating the "it works on my machine" problem. This chapter details the entire process, from the setup of the test environment to the execution of various testing levels and the management of defects. A visual representation of our high-level automated testing process is shown in Figure ??.

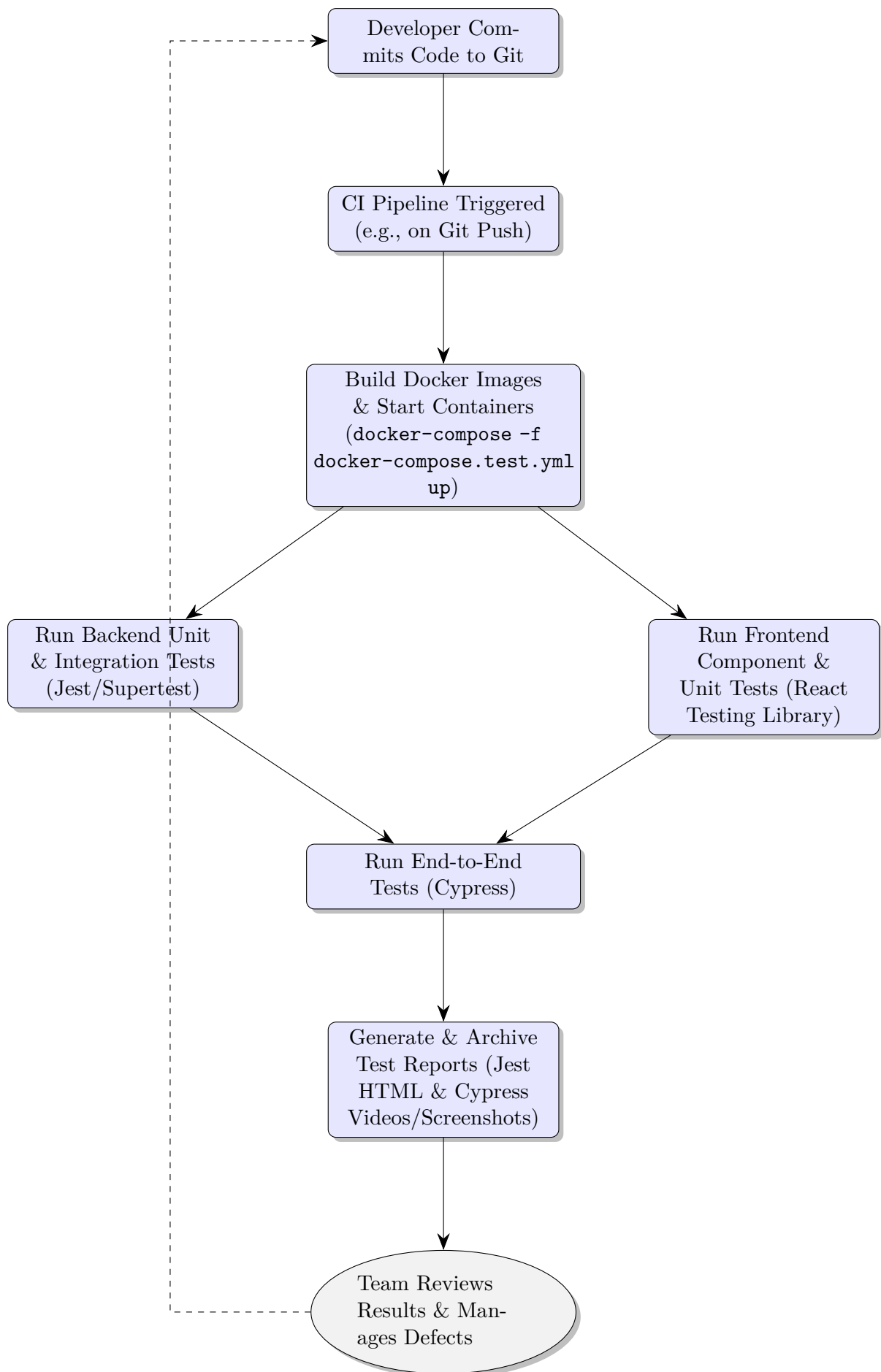


Figure 3.1: High-Level Automated Testing Process Flowchart

3.2 Test Environment and Configuration

A consistent and reproducible test environment is the cornerstone of our testing strategy. We use Docker and Docker Compose to define and manage the application stack, ensuring that tests are executed in an environment that mirrors production as closely as possible. This minimizes environment-specific defects and guarantees that a test that passes on one developer's machine will also pass in the CI pipeline.

The entire test environment is orchestrated by the `docker-compose.test.yml` file and configured via the `.env.test` file. This setup allows any team member to spin up the full test stack with a single command (`docker-compose -f docker-compose.test.yml up -build`). The services defined are:

- **server-test**: The Node.js backend application. For API and integration tests, it is configured via `.env.test` to connect to an in-memory MongoDB server. This provides significant advantages: test isolation (no data leakage between tests), speed (no disk I/O), and simplicity (no need for manual database cleanup).
- **mongo-test**: The MongoDB database. While most automated tests use the in-memory version, this service is available for specific tests that may require validation against a persistent database instance.
- **client-test**: The React frontend application, serving the user interface against which end-to-end tests are run.
- **cypress-e2e**: A dedicated container that runs the Cypress test runner for executing end-to-end tests against the client and server. It is configured to communicate with the other services within the Docker network.

This containerized architecture, illustrated in Figure ??, is paramount for maintaining testing integrity across all development and deployment stages.

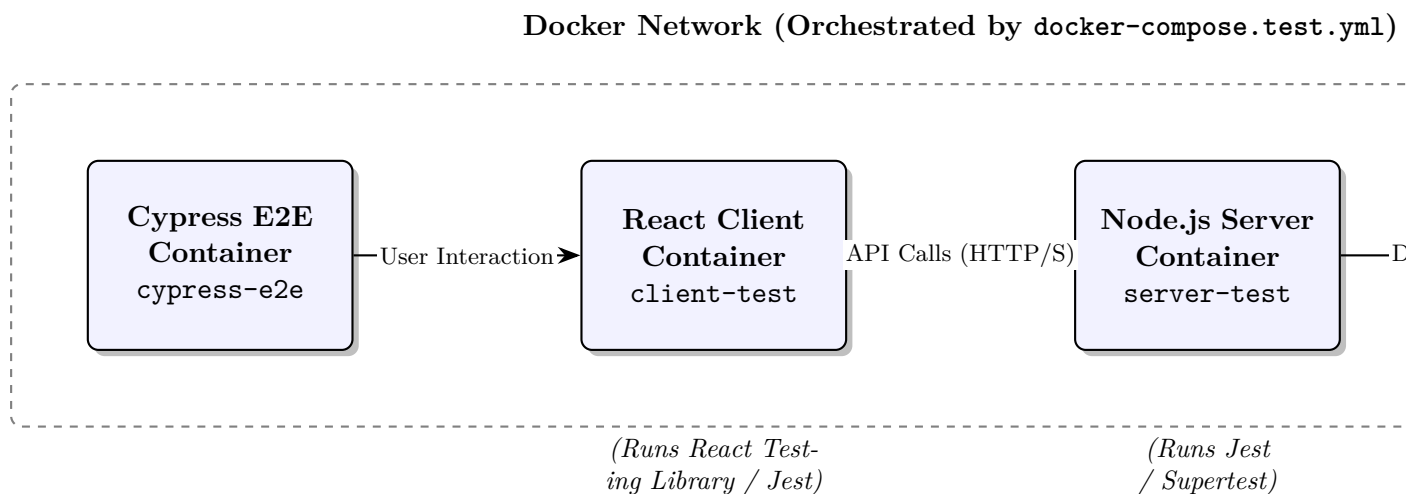


Figure 3.2: Docker-Based Test Environment Architecture

3.3 Testing Levels and Automation Frameworks

Our testing process is composed of several distinct levels, each targeting a different part of the application and utilizing specialized tools and techniques to provide comprehensive coverage.

3.3.1 Unit Testing

Unit tests form the foundation of our testing pyramid, focusing on the smallest individual components (units) of the system in isolation.

- **Backend (Server):** We use the **Jest** framework to test individual controllers, models, and utility functions. Mocks and stubs are used extensively to isolate units from external dependencies. For example, in `bookController.test.js`, the `Book` model is mocked to simulate database interactions, ensuring the test validates only the controller's internal logic. Similarly, tests for models themselves (e.g., `book.test.js`) verify schema validations and default values without needing a database connection.
- **Frontend (Client):** We use **React Testing Library** with Jest to test individual React components and custom hooks. Component tests verify that they render correctly given specific props and that they respond as expected to simulated user interactions. For instance, tests for `LoginForm.js` check for the presence of input fields and validate that clicking the submit button triggers the correct callback function. We also test utility functions like `formatTime.js` and `formatNumber.js` to ensure data is displayed correctly across the UI.

```
1 // From: server/__tests__/unit/controllers/bookController.test.js
2
3 const bookController = require('../../../controllers/bookController');
4 const Book = require('../../../models/book');
5
6 // Mock the Book model to isolate the controller logic
7 jest.mock('../../../models/book');
8
9 describe('Book Controller - getBookById', () => {
10   it('should return a book with status 200 if a valid ID is passed',
11     async () => {
12       const mockBook = { _id: '60c72b2f9b1d8c001f8e4c6d', title: 'Test
13       Book', author: 'Author Name' };
14       Book.findById.mockResolvedValue(mockBook);
15
16       const req = { params: { id: '60c72b2f9b1d8c001f8e4c6d' } };
17       const res = {
18         status: jest.fn().mockReturnThis(),
19         json: jest.fn()
20       };
21
22       await bookController.getBookById(req, res);
23
24       expect(res.status).toHaveBeenCalledWith(200);
25       expect(res.json).toHaveBeenCalledWith(mockBook);
26     });
27 });
```

Listing 3.1: Example of a Backend Unit Test for `bookController.js`

3.3.2 Integration Testing

Integration tests focus on verifying the interactions and data flow between different parts of the system.

- **Backend API Testing:** We use **Jest** and **Supertest** to perform integration testing on our API endpoints. These tests are critical for ensuring that controllers, models, and services work together as expected. The tests run against a live server instance connected

to an in-memory MongoDB database to ensure a clean state for each test suite. We test not only basic CRUD operations but also complex business workflows. For example, `borrowal.api.test.js` tests the full lifecycle of borrowing and returning a book, while `bookReferentialIntegrity.api.test.js` ensures that a book cannot be created with a non-existent author.

- **Frontend Component Integration:** At the component level, we test how multiple components collaborate to deliver a piece of functionality. For example, on the `AuthorPage.jsx`, we test the integration between the main data table, the `AuthorDialog.jsx` (for adding/editing), and the search/filter toolbar, ensuring that actions performed in one component correctly update the state and rendering of the others.

3.3.3 System (End-to-End) Testing

System testing validates the complete, integrated application workflow from a user's perspective. We use **Cypress** for our end-to-end (E2E) tests because of its reliability and powerful debugging features. These tests simulate real user scenarios by interacting with the application through the browser, just as a user would.

Cypress tests cover critical user journeys derived from the use cases in the SRS (Section 7), such as:

- **UC-001 (User Login):** As shown in `authentication_authorization.cy.ts`, a test logs in as an admin and verifies access to the dashboard.
- **UC-002 & UC-004 (Entity Management):** The `entity_management.cy.ts` script simulates an admin creating a new author, then a new genre, and finally a new book that uses the newly created author and genre.
- **UC-003 (Borrow a Book):** The `full_lifecycle.cy.ts` test simulates a member logging in, searching for a book, borrowing it, and then viewing it in their borrowal history.
- **Authorization:** Tests also explicitly verify access control rules, such as ensuring a non-admin user receives a "403 Forbidden" or is redirected when attempting to access administrative API endpoints or frontend routes.

To manage test data, Cypress uses fixtures (e.g., `user-data.json`) to provide consistent user credentials and other data inputs for the tests.

```
1 // From: e2e/cypress/e2e/authentication_authorization.cy.ts
2
3 describe('Authentication and Authorization', () => {
4   it('should allow an admin user to log in and see the main dashboard', ()
5     => {
6     // Visit the login page
7     cy.visit('/login');
8
9     // Use test data from a fixture file
10    cy.fixture('user-data.json').then((userData) => {
11      cy.get('input[name="email"]').type(userData.adminUser.email);
12      cy.get('input[name="password"]').type(userData.adminUser.password);
13    });
14
15    // Click the submit button
16    cy.get('button[type="submit"]').click();
17
18    // Assert that the URL has changed to the dashboard
19    cy.url().should('include', '/dashboard/app');
20
21    // Assert that a key element of the dashboard is visible to confirm
    // success
22    cy.contains('h4', 'Welcome back').should('be.visible');
```

```
22     });  
23   });
```

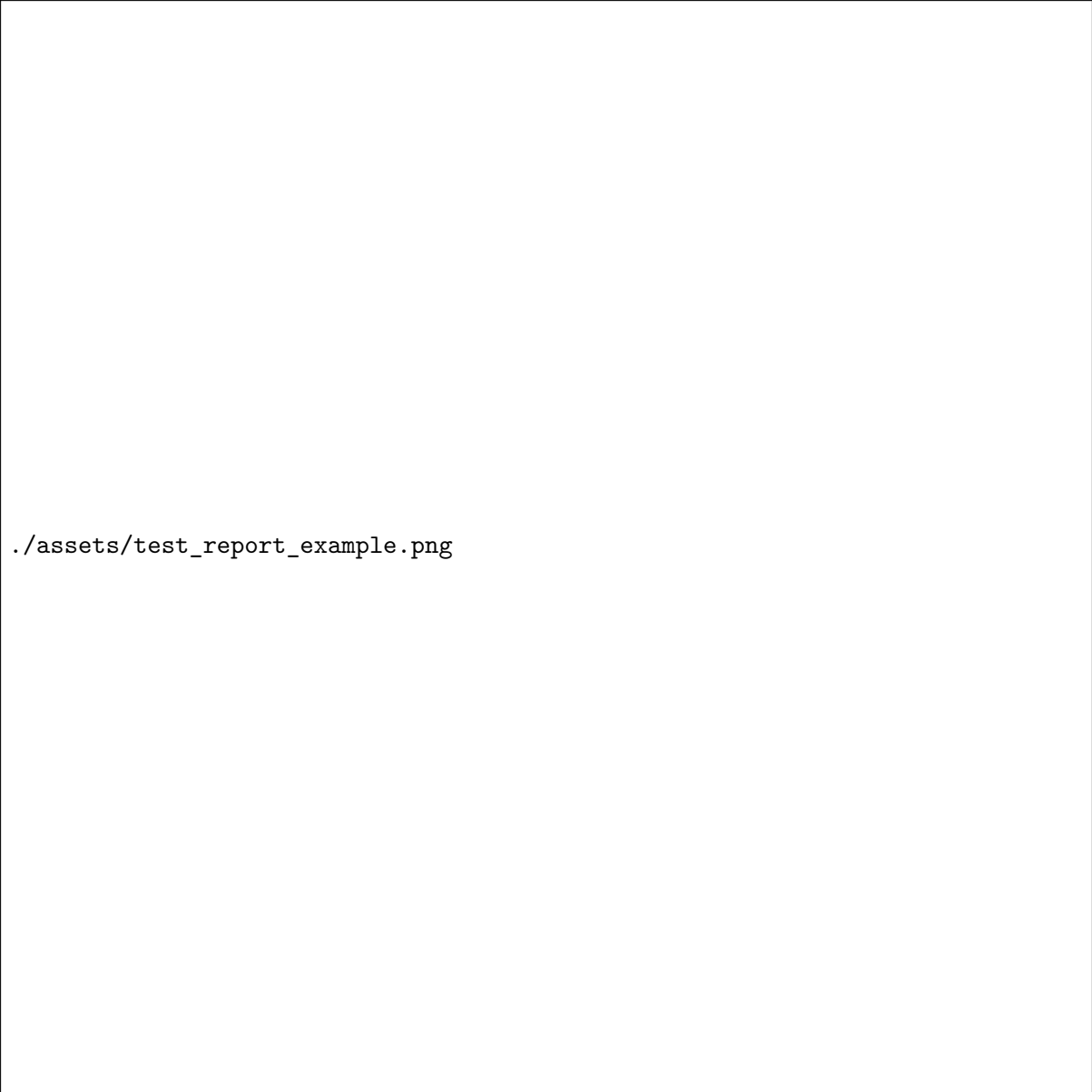
Listing 3.2: Example of a Cypress E2E Test for User Login

3.4 Test Execution and Reporting

Test execution is streamlined and automated through npm scripts defined in the respective `package.json` files, enabling easy execution both locally and in a CI/CD pipeline.

- **Backend Tests:** Running `npm test` in the `/server` directory executes all Jest-based unit and integration tests.
- **Frontend Tests:** Running `npm test` in the `/client` directory executes all React Testing Library component tests.
- **E2E Tests:** Running `npm run cypress:run` in the `/e2e` directory executes the full suite of Cypress end-to-end tests in headless mode.

Upon completion, the test frameworks generate detailed reports. Jest is configured with `jest-html-reporters` to produce a comprehensive HTML report (located in `server/test-report/`), which provides a shareable overview of test suite results, pass/fail status, execution times, and code coverage metrics. Code coverage is particularly important as it helps the team identify areas of the application that are not covered by tests. Cypress automatically generates video recordings of each test run and detailed screenshots upon failure, which are invaluable for quickly diagnosing the root cause of an issue.



`./assets/test_report_example.png`

Figure 3.3: Example of an HTML Test Report from Jest

3.5 Defect Management

A formal defect management process is essential for tracking and resolving issues discovered during testing. Our process ensures that no bug is lost and that every issue is triaged, prioritized, and resolved in a timely manner. While a dedicated tool like Jira or Trello is recommended, this process can also be managed using GitHub Issues.

The defect lifecycle is as follows:

1. **Discovery and Logging:** When a test fails or a bug is found during manual testing, the tester logs a new defect. The defect report includes a clear title, a detailed description of the issue, steps to reproduce it, the expected result vs. the actual result, and any relevant artifacts like screenshots, logs, or video recordings from Cypress.
2. **Triage and Prioritization:** The Test Manager and technical lead review new defects to assess their impact and urgency. Defects are assigned a priority (e.g., Critical, High,

Medium, Low) based on factors like severity, scope of impact, and risk to the project timeline.

3. **Assignment:** The defect is assigned to the appropriate developer for resolution.
4. **Resolution:** The developer fixes the bug, commits the code, and marks the defect as "Resolved." The fix should ideally include a new regression test that specifically targets the bug to prevent it from recurring.
5. **Verification:** The tester re-runs the failing test in the updated environment to verify that the fix has resolved the issue and has not introduced any new regressions. If the fix is confirmed, the defect is closed. If not, it is reopened and reassigned to the developer.

This structured workflow ensures accountability and provides clear visibility into the health of the application throughout the development process.